SOFTWARE DEV

MERN STACK DEVELOPMENT REPORT

WEEK 3 FOCUS

Advanced DOM Manipulation & Event Delegation

Muneeb Ur Rehman

MERN STACK INTERN

1. Task Objective

The primary goal of this task was to build a dynamic list User Interface (UI) that allows users to add tasks, mark them as completed, and delete them. The core technical requirement was to utilize **Event Delegation**—attaching only a single event listener to a parent container to handle interactions across multiple dynamically generated child elements.

2. Core Concepts Applied

- **Event Bubbling:** Leveraging the natural behavior of the DOM where events triggered on a child element (like a button) "bubble up" to their parent elements.
- **Event Delegation:** Instead of assigning individual event listeners to every list item (<li>) or delete button, a single listener was assigned to the parent Unordered List (<ul>).
- **Dynamic DOM Manipulation:** Using JavaScript to actively change the structure of the webpage without reloading.
    - document.createElement() and innerHTML to construct new nodes.
    - appendChild() to render new tasks.
    - remove() to delete tasks from the DOM.

3. Expected Output & User Experience

When the HTML file is opened in a web browser, the user experiences the following flow:

- **Initial State:** A centered, clean white card titled "My Tasks" appears against a light gray background. It contains an input bar, a blue "Add" button, and a sample task ("Learn Event Bubbling") with a red "Delete" button.
- **Adding a Task:** When the user types "Master Event Delegation" and clicks "Add", the input field instantly clears. A new row flawlessly appears at the bottom of the list containing the new text and its own "Delete" button.
- **Marking as Complete:** When the user clicks directly on the text of any task, the row's background turns a faint gray, and the text is crossed out (strikethrough) to indicate completion. Clicking it a second time reverts it to normal.
- **Deleting a Task:** When the user clicks the red "Delete" button on any row, that specific task instantly vanishes from the list, and the remaining tasks shift up to fill the space.

4. Technical Implementation

A. User Interface (UI) Design

The UI was built using a modern, card-based layout centered on the screen.

- **Flexbox** was utilized to ensure the task input field and the "Add" button sit neatly next to each other, and to align the task text and the "Delete" button inside each list item.
- **Visual Feedback:** CSS transitions and hover effects were added to buttons and list items to improve the user experience (UX). A specific .completed class was styled to strike through the text and change the background color.

B. Adding Tasks Dynamically

When the user clicks the "Add" button:

- The application captures the text from the input field and trims whitespace.
- A new <li> element is created in memory.
- Template literals (innerHTML) are used to inject both the task text (wrapped in a) and a <button> for deletion into the <li>.
- The new element is appended to the parent <ul>, and the input field is cleared.

C. Event Delegation Logic

A single click event listener was attached to the id="taskList" element. The logic relies on event.target to determine exactly what the user clicked and routes the action accordingly:

User Action	Code Logic (event.target)	Result
Clicks "Delete" Button	Checks if target has class .btn-delete	Finds parent using .closest('li') and calls .remove().
Clicks List Item	Checks if target tagName === 'LI'	Toggles the .completed CSS class on the clicked element.

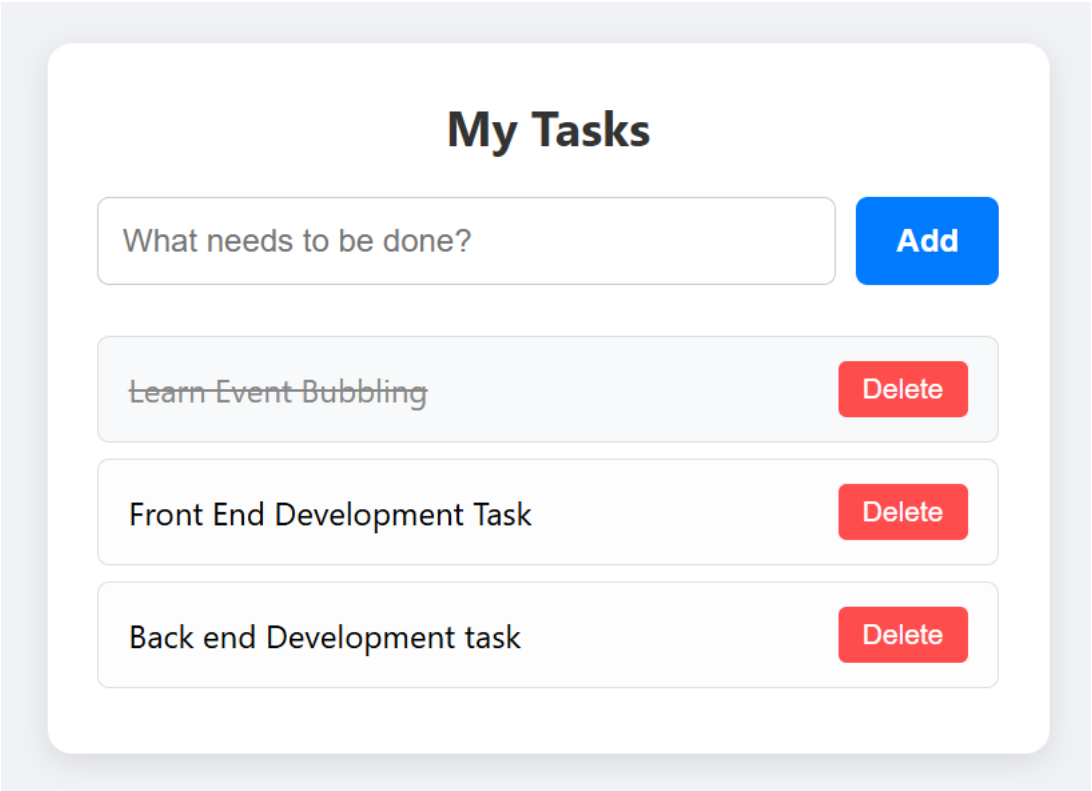
D. Overcoming Edge Cases

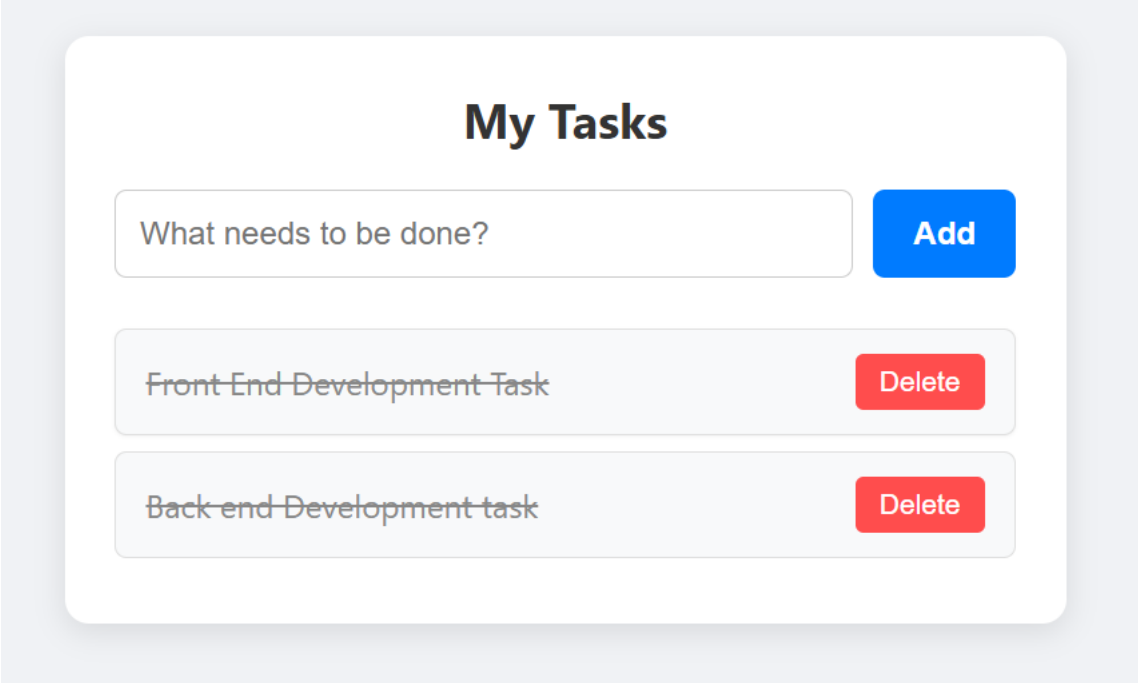
A common issue with event delegation occurs when a user clicks the *text inside* the list item rather than the list item itself. This was cleanly resolved using a CSS property:

- pointer-events: none; was applied to the .task-text span. This forces the browser to ignore clicks on the text and register the click directly on the parent <li> underneath it, ensuring the "mark as complete" logic works flawlessly every time.

5. Conclusion

The task was successfully completed according to the given instructions. By implementing Event Delegation, the application is highly performant and scalable. Whether the list contains two tasks or two thousand, the memory footprint remains low because only one event listener is required to manage all interactions.





https://github.com/955muneeb/Glaxit-internship/tree/main/week-3/day-3